Distance Sampling

Ellen Bledsoe

2026-04-20

Distance Sampling Lab

In this lab, we will use the data we collected from the first lab in this module, a demonstration of distance sampling using clear and green class beads. We will work through the code for completing a distance sampling analysis.

Set-Up

We need to load the package we will need to complete this lab, which is `unmarked`.

```
# Load unmarked
library(unmarked)
```

We also need to load our data. I've compiled it for you already, and it should be in your list of files, called `Distance_Data_Spring2026.csv`.

```
beads <- read.csv("../data_raw/Distance_Data_Spring2026.csv")
```

Before we begin any analyses, let's make sure our data looks alright.

```
str(beads)    # View the structure of the dataframe
```

```
## 'data.frame':    494 obs. of  4 variables:
##  $ GroupID    : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ BeadColor  : chr  "Clear" "Clear" "Clear" "Clear" ...
##  $ Distance_in: num  11 1 14.8 5.5 3 ...
##  $ Transect_m : num  26.5 26.5 26.5 26.5 26.5 26.5 26.5 26.5 26.5 26.5 ...
```

```
head(beads)   # First few observations
```

```
##   GroupID BeadColor Distance_in Transect_m
## 1       1     Clear       11.00       26.5
## 2       1     Clear        1.00       26.5
## 3       1     Clear       14.75       26.5
## 4       1     Clear        5.50       26.5
## 5       1     Clear        3.00       26.5
## 6       1     Clear       22.50       26.5
```

Prepping the Data

First, we need to convert all of our data from inches into meters because those are the measurements `unmarked` is expecting.

You won't need to do this in your assignment, thankfully, because those values are already in meters.

```r
beads$Distance_m <- beads$Distance_in / 39.37
```

Since we have 2 different colors of beads, we need to create two different data frames. *You won't need to do this in your assignment, either!*

```r
clear <- beads[beads$BeadColor == "Clear", ]
green <- beads[beads$BeadColor == "Green", ]
```

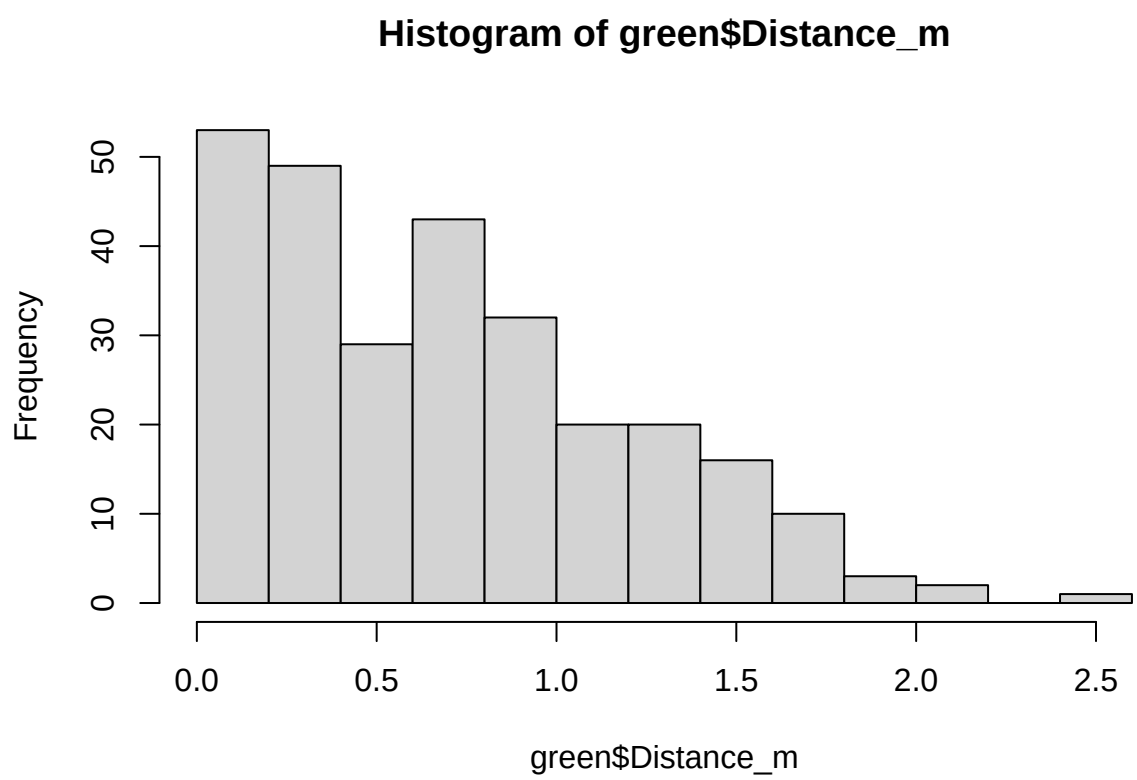
Take a look at the environment to see how many rows the `clear` and `green` data frames have. The `green` data frame has more observations than the `clear`. Is that expected? Why or why not?

Green Beads

We will start our analysis with the green bead data.

Let's check out our distribution of detection distances. We can use the `hist()` function to do this.

```r
hist(green$Distance_m)
```



Based on the histogram, I'll suggest we truncate at 2.25 m. Let's set this value so we can refer to it later.

```r
# Set truncation distance to eliminate extreme observations
trunc <- 2.25
```

We can set "cut points" for which values to use to bin our distance data in our model; in our case, we want every quarter meter.

This line of code creates a vector that will have values every quarter meter (`by = 0.25`), starting at 0 and running through the value we set for truncation (`trunc`).

```r
distance_bins <- seq(0, trunc, by = 0.25)
```

`unmarked` Data Prep

As we've seen before, the `unmarked` package likes to have data set up in a very specific way and has specific functions for this.

To get our detection functions and density estimates, we will use the `formatDistData` to get the data into the correct format.

```r
# create an unmarked data frame
# ignore any "Warnings"; if you see "Error," something has gone wrong

green_binned <- formatDistData(distData = green,       # data frame with the data points
                        distCol = "Distance_m",        # column that holds the distance values
                        transectNameCol = "GroupID",   # column that holds the transect groups
                        dist.breaks = distance_bins)   # vector of bin groupings for distances
```

```
## Warning in formatDistData(distData = green, distCol = "Distance_m",
## transectNameCol = "GroupID", : formatDistData will be deprecated in the next
## version.
```

```
## Warning in formatDistData(distData = green, distCol = "Distance_m",
## transectNameCol = "GroupID", : The transects were converted to a factor
```

```r
green_binned
```

```
##    [0,0.25] (0.25,0.5] (0.5,0.75] (0.75,1] (1,1.25] (1.25,1.5] (1.5,1.75]
## 1         2          6          5        5        1          0          0
## 2         3          5          7        6        2          3          0
## 3         6          5          1        2        4          7          3
## 4         5          7          2        3        4          0          1
## 5         2          3          9        5        0          1          3
## 6         3          4          2        1        5          0          2
## 7         2          4          4        3        2          0          1
## 8         6         10          4        2        3          2          2
## 9         9          6          0        1        0          2          0
## 10        5          4          4        7        7          1          4
## 11        9          8          2        2        1          3          0
## 12        5          5          3        2        2          0          0
##    (1.75,2] (2,2.25]
## 1         0        0
## 2         1        0
## 3         0        0
## 4         0        0
## 5         0        0
## 6         0        0
## 7         0        0
## 8         0        0
## 9         0        0
## 10        2        2
## 11        0        0
## 12        0        0
```

Next, we need to assemble data into the format required by `unmarked`, called an "unmarked frame."

Before we do that, we need to create a new data frame that includes the length of transect that each group

walked so we have a measure of survey effort.

We will use the `unique()` function, which returns only one of each value in a column. We want the unique values from two of our columns: the group and the transect length.

```
# create a data frame that has a column for each group id and a column for the length that each group s

# one row for each set of values from the "GroupID" and "Transect_m" columns
transect_length <- unique(green[c(1,4)])
transect_length
```

```
##     GroupID Transect_m
## 13        1       26.5
## 47        2       26.5
## 107       3       26.5
## 157       4       26.5
## 189       5       26.5
## 228       6       26.5
## 266       7       26.5
## 296       8       26.5
## 341       9       26.5
## 379      10       26.5
## 424      11       26.5
## 478      12       26.5
```

This dataframe we just created will be part of the information we give to the `unmarked` function to create the "unmarked frame."

We've seen and used a function similar before in our occupancy model assignment. Today, we are using `unmarkedFrameDS`, with the `DS` meaning "distance sampling."

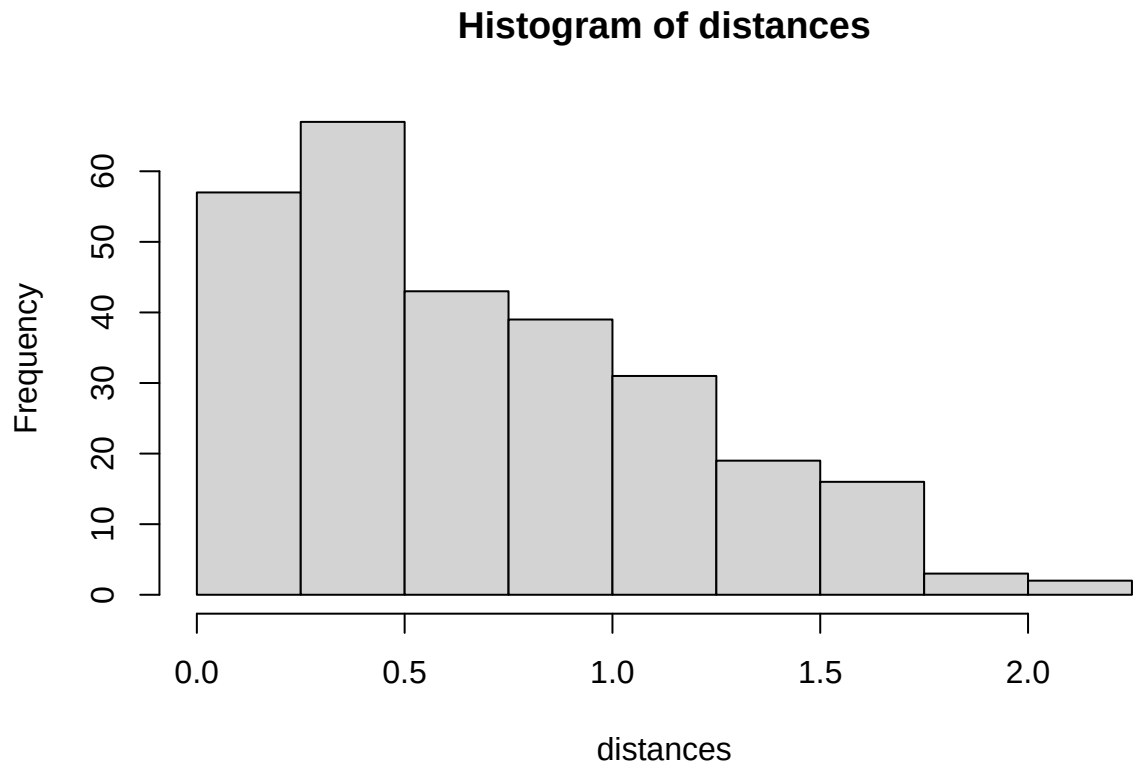
```
# join binned dataframe with transect length dataframe
UMF_green <- unmarkedFrameDS(y = green_binned,        # binned data
                       survey = "line",             # type of survey conducted (line or point)
                       tlength = transect_length$Transect_m,  # column w/ each transect length
                       dist.breaks = distance_bins, # vector of how distances are binned
                       unitsIn = "m")

UMF_green   # prepared data
```

```
## Data frame representation of unmarkedFrame object.
##    y.1 y.2 y.3 y.4 y.5 y.6 y.7 y.8 y.9
## 1    2   6   5   5   1   0   0   0   0
## 2    3   5   7   6   2   3   0   1   0
## 3    6   5   1   2   4   7   3   0   0
## 4    5   7   2   3   4   0   1   0   0
## 5    2   3   9   5   0   1   3   0   0
## 6    3   4   2   1   5   0   2   0   0
## 7    2   4   4   3   2   0   1   0   0
## 8    6  10   4   2   3   2   2   0   0
## 9    9   6   0   1   0   2   0   0   0
## 10   5   4   4   7   7   1   4   2   2
## 11   9   8   2   2   1   3   0   0   0
## 12   5   5   3   2   2   0   0   0   0
```

Let's check the distribution of detection distances to be used for analysis. These should *not* include the values that we truncated.

```r
hist(UMF_green)
```



Models

Now that our data is in the correct format for `unmarked`, we can use the `distsamp` function from `unmarked` to create our 4 types of models: half normal, hazard rate, uniform, and negative exponential.

Our options for the `unitsOut` argument is either hectares (ha) or kilometer (km). We will use hectares. This means that the density we calculate will be the density of beads per hectare.

```r
HN   <- distsamp(~1 ~1, UMF_green, keyfun = "halfnorm", output = "density", unitsOut = "ha")
HR   <- distsamp(~1 ~1, UMF_green, keyfun = "hazard", output = "density", unitsOut = "ha")
Unif <- distsamp(~1 ~1, UMF_green, keyfun = "uniform", output = "density", unitsOut = "ha")
Exp  <- distsamp(~1 ~1, UMF_green, keyfun = "exp", output = "density", unitsOut = "ha")
```

Model Selection

Which model should we choose? We will use AIC again to help us figure it out. We want the model with the *lowest* AIC value.

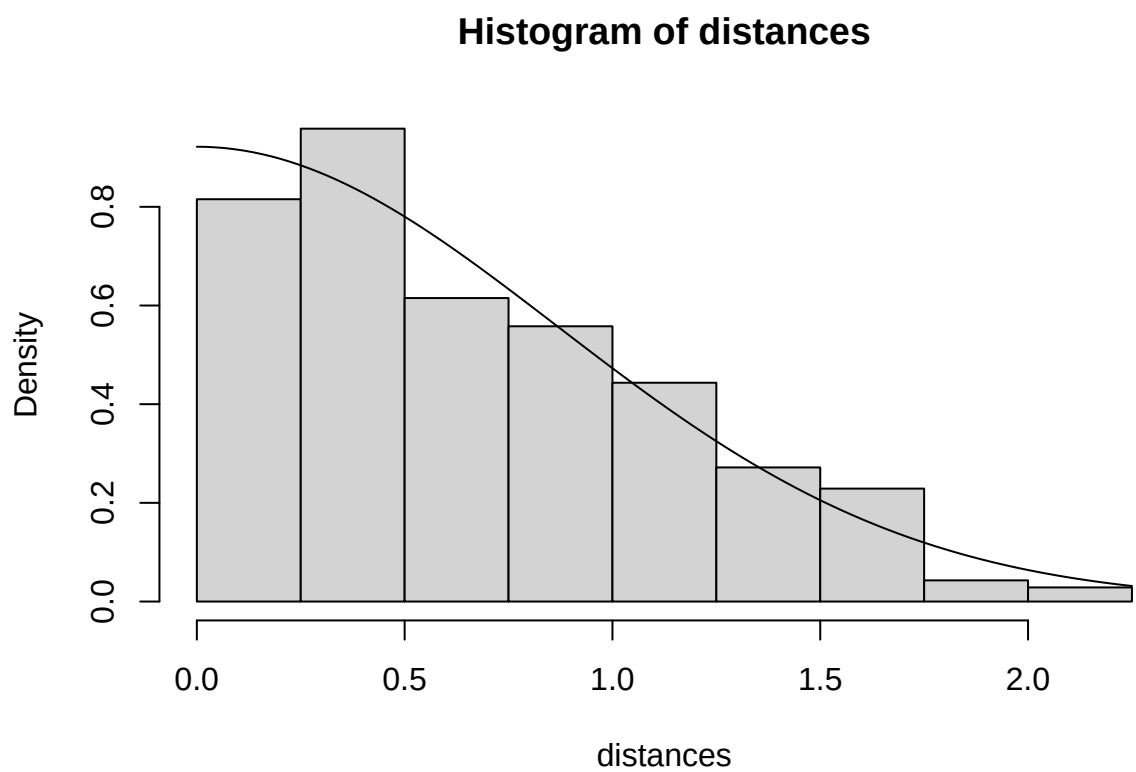
```r
models <- fitList('Half Normal' = HN,
                  'Hazard Rate' = HR,
                  'Uniform'     = Unif,
                  'Exponential' = Exp)
modSel(models)
```

```
##             nPars    AIC  delta   AICwt cumltvWt
## Half Normal     2 383.85   0.00 9.7e-01     0.97
```

```
## Hazard Rate     3 391.11   7.27 2.6e-02     1.00
## Exponential     2 399.53  15.68 3.8e-04     1.00
## Uniform         1 532.18 148.34 6.0e-33     1.00
```

Let's plot what the half-normal model looks like with our data.

```r
hist(HN)
```



Density Estimate

Now that we've chosen our top model (half normal, in this case), we can get our density estimate!

There is a handy function that we can use to pull the density estimate out of the model: `backTransform()`

```r
density <- backTransform(HN, type = "state") # Density estimate (no./ha)
density
```

```
## Backtransformed linear combination(s) of Density estimate(s)
##
##  Estimate  SE LinComb (Intercept)
##      4053 309    8.31           1
##
## Transformation: exp
```

We can also calculate the confidence intervals for our parameter estimate.

```r
exp(confint(HN, type = "state"))  # CI for density
```

```
##             0.025    0.975
## lam(Int) 3490.352 4705.683
```

Convert to Our Sample Area

You won't need to do this in your assignment—we will just leave all of our density estimates in the numbers per hectare. For our bead data, however, we definitely didn't sample a whole hectare (100m x 100m).

To see how close our estimate was to the actual number of beads we put outside, we need to convert the density in hectares to the density of our survey area.

Our first step is to calculate our *effective strip width*.

```
# density estimate
density_est <- density@estimate
density_est
```

```
## [1] 4052.714
```

```
# standard deviation
sigma <- backTransform(HN, type = "det")@estimate
sigma
```

```
## [1] 0.8655613
```

```
# calculate the effective strip width
esw <- integrate(gxhn, lower = 0, upper = 2.25, sigma = sigma)$value
esw
```

```
## [1] 1.074691
```

We can then calculate the area surveyed

```
# Our survey area (m) = [12 groups x (2*esw*L)]
area <- (12 * (2 * esw * 26.5))
area
```

```
## [1] 683.5037
```

```
# One hectare = 100 m x 100 m
ha <- 100 * 100 # 10,000 m^2
ha
```

```
## [1] 10000
```

```
# Proportion of hectare we surveyed
surveyed <- area/ha
surveyed
```

```
## [1] 0.06835037
```

Finally, if we multiple our density estimate by the amount of area we surveyed, we will get the density of beads for the entirety of our sample area (same as abundance!)

```
# D-hat * surveyed
density_est * surveyed
```

```
## [1] 277.0045
```

I put out 150 green beads, so this is off by a bit! That's ok, though.

We did *repeated* surveys in the same area and did not take that into account in our model. If we were going to accurately analyze our data, we would definitely need to do that.